

Animal Deterrent System

Using ESP32-CAM, Blynk IoT, and AI-based Detection with Email Alerts

Project Report

Prepared by: Girdhari Chaturvedi
Diploma - Electronics & Telecommunication (ET&T;)
Raipur, Chhattisgarh

1. Project Overview

This project builds a low-cost, WiFi-connected Animal Deterrent system using an ESP32-CAM module. The system streams live video, can capture snapshots, sounds a buzzer and flashes an LED to scare away animals or unwanted intruders, and sends push notifications through the Blynk IoT platform. An additional PC-based Python component uses a pre-trained AI object-detection model (YOLOv8) to automatically recognise a person or animal in the camera feed and email an alert with a photo attached.

How the system works

- ESP32-CAM captures live video and serves it over WiFi as a web page (with a live stream, a photo-capture button, and a manual test-alert button).
- When triggered, the ESP32-CAM turns on a buzzer and flash LED, and sends a push notification via the Blynk IoT app.
- A Python script running on a PC (same WiFi network) periodically fetches a photo from the ESP32-CAM, runs it through a pre-trained YOLOv8 detection model, and — if a person or animal is recognised — emails an alert with the photo attached.

2. Components Required

#	Component	Purpose / Notes
1	ESP32-CAM (AI-Thinker board)	Main microcontroller with built-in OV2640 camera
2	FTDI Programmer / USB-TTL converter (5V)	Used only to upload code to the ESP32-CAM
3	Buzzer (5V)	Sound-based deterrent, connected to GPIO 13
4	Jumper wires (Male-Female / Male-Male)	For all connections
5	Breadboard	For prototyping connections
6	5V power adapter / power bank	Stable field power supply (for permanent deployment)
7	PC / Laptop	Runs Arduino IDE (for programming) and the Python detection script

Note: The ESP32-CAM's built-in Flash LED (GPIO 4) is used automatically by the camera hardware and does not need separate wiring.

3. Circuit Connections

3.1 Programming Mode (Uploading code via FTDI)

FTDI / USB-TTL Pin	ESP32-CAM Pin	Notes
5V	5V	Use 5V, NOT 3.3V — camera needs higher peak current
GND	GND	Common ground
TX	U0R (RX)	Crossed connection
RX	U0T (TX)	Crossed connection
—	GPIO 0 → GND	Jumper wire; enables flashing mode. Remove after upload.

After clicking Upload in Arduino IDE, press the ESP32-CAM's RESET button once when "Connecting..." appears, to enter flashing mode.

3.2 Buzzer (Deterrent Hardware)

Buzzer Pin	ESP32-CAM Pin	Notes
+ (Positive)	GPIO 13	Controlled in software (HIGH = ON)
- (Negative)	GND	Common ground

3.3 Normal Run Mode (after upload)

- Remove the GPIO 0 - GND jumper wire.
- Press the RESET button so the board boots normally (not in flashing mode).
- Power can now come from the FTDI's 5V pin, or an independent 5V adapter / power bank for permanent field deployment.

Important: The camera module requires a stable 5V supply during initialisation (it draws a current spike). A weak or 3.3V supply is the most common cause of camera initialisation failures.

4. Software Setup — Arduino IDE & Blynk

4.1 Install Arduino IDE and the ESP32 board package

- 1 Download and install the Arduino IDE from [arduino.cc](https://www.arduino.cc).
- 2 Go to File > Preferences and add this URL under "Additional Board Manager URLs":
- 3 https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
- 4 Go to Tools > Board > Boards Manager, search "esp32", and install the Espressif package.

4.2 Select board settings

Setting	Value
Board	AI Thinker ESP32-CAM
Upload Speed	115200
Flash Mode	DIO
Partition Scheme	Huge APP (3MB No OTA/1MB SPIFFS)
Port	COM port where the FTDI is connected

4.3 Install the Blynk library

In Arduino IDE: Tools > Manage Libraries (Ctrl+Shift+I), search "Blynk", and install **Blynk by Volodymyr Shymanskyi**.

4.4 Create a Blynk Template and Device

- 1 Sign up / log in at blynk.cloud.
- 2 Go to Developer Zone > Templates > New Template. Set Name: "Animal Deterrent", Hardware: ESP32, Connection Type: WiFi.
- 3 Inside the template, go to the Events tab and add a new event: Name "Animal Alert", Event Code "animal_alert" (lowercase). Enable push notifications for this event.
- 4 Go to Devices > New Device > From Template, select the template, and create the device.
- 5 Copy the three lines shown (Template ID, Template Name, Auth Token) — these go at the very top of the Arduino code.
- 6 Install the Blynk IoT app on your phone (Play Store / App Store), log in with the same account, and allow notifications.

5. ESP32-CAM Code

This is a single self-contained .ino file — no separate board_config.h, camera_pins.h, or app_httpd.cpp files are needed. It combines: live MJPEG video streaming, a full-screen web viewer page, a photo-capture endpoint, a manual test-alert button, and Blynk push-notification integration.

Before uploading: fill in BLYNK_TEMPLATE_ID, BLYNK_TEMPLATE_NAME, BLYNK_AUTH_TOKEN (from Blynk.cloud) and your WiFi ssid / password near the top of the file.

```
// =====
// ANIMAL DETERRENT - ESP32-CAM (AI-Thinker)
// Single-file version - NO external files needed
// (no board_config.h / camera_pins.h / app_httpd.cpp required)
// =====

// ---- Blynk credentials: must stay at the very top ----
#define BLYNK_TEMPLATE_ID "YOUR_TEMPLATE_ID" // <-- Blynk.cloud se copy karein
#define BLYNK_TEMPLATE_NAME "YOUR_TEMPLATE_NAME" // <-- Blynk.cloud se copy karein
#define BLYNK_AUTH_TOKEN "YOUR_AUTH_TOKEN" // <-- Blynk.cloud se copy karein

#include <Arduino.h>
#include "esp_camera.h"
#include <WiFi.h>
#include "esp_http_server.h"
#include "img_converters.h"
#include <BlynkSimpleEsp32.h>

// ---- WiFi credentials ----
const char *ssid = "Infinix GT 30 Pro";
const char *password = "YOUR_WIFI_PASSWORD";

// ---- AI-Thinker ESP32-CAM pin map ----
#define PWDN_GPIO_NUM 32
#define RESET_GPIO_NUM -1
#define XCLK_GPIO_NUM 0
#define SIOD_GPIO_NUM 26
#define SIOC_GPIO_NUM 27
#define Y9_GPIO_NUM 35
#define Y8_GPIO_NUM 34
#define Y7_GPIO_NUM 39
#define Y6_GPIO_NUM 36
#define Y5_GPIO_NUM 21
#define Y4_GPIO_NUM 19
#define Y3_GPIO_NUM 18
#define Y2_GPIO_NUM 5
#define VSYNC_GPIO_NUM 25
#define HREF_GPIO_NUM 23
#define PCLK_GPIO_NUM 22

// ---- Deterrent hardware ----
#define BUZZER_PIN 13 // External buzzer

httpd_handle_t stream_httpd = NULL;

// Forward declaration (defined further below, used by trigger_handler)
void triggerDeterrent();

// ---- MJPEG stream handler ----
// Camera captures RGB565 (this sensor doesn't init reliably in JPEG mode),
// so each frame is converted to JPEG in software before sending.
static esp_err_t stream_handler(httpd_req_t *req) {
    camera_fb_t *fb = NULL;
    esp_err_t res = ESP_OK;
    size_t _jpg_buf_len = 0;
    uint8_t *_jpg_buf = NULL;
    char part_buf[64];

    static const char *STREAM_BOUNDARY = "\r\n--1234567890000000000000987654321\r\n";
    static const char *STREAM_PART = "Content-Type: image/jpeg\r\nContent-Length: %u\r\n\r\n";

    res = httpd_resp_set_type(req, "multipart/x-mixed-replace;boundary=1234567890000000000000987654321");
    if (res != ESP_OK) return res;

    while (true) {
        fb = esp_camera_fb_get();
        bool jpg_needs_free = false;

        if (!fb) {
            Serial.println("Camera capture failed");
            res = ESP_FAIL;
        } else if (fb->format == PIXFORMAT_JPEG) {
            _jpg_buf = fb->buf;
            _jpg_buf_len = fb->len;
        }
```

```

    } else {
        bool converted = frame2jpg(fb, 80, &_jpg_buf, &_jpg_buf_len);
        esp_camera_fb_return(fb);
        fb = NULL;
        if (!converted) {
            Serial.println("JPEG conversion failed");
            res = ESP_FAIL;
        } else {
            jpg_needs_free = true;
        }
    }
}

if (res == ESP_OK) {
    size_t hlen = snprintf(part_buf, 64, STREAM_PART, _jpg_buf_len);
    res = httpd_resp_send_chunk(req, part_buf, hlen);
}
if (res == ESP_OK) {
    res = httpd_resp_send_chunk(req, (const char *)_jpg_buf, _jpg_buf_len);
}
if (res == ESP_OK) {
    res = httpd_resp_send_chunk(req, STREAM_BOUNDARY, strlen(STREAM_BOUNDARY));
}

if (fb) {
    esp_camera_fb_return(fb);
    fb = NULL;
}
if (jpg_needs_free && _jpg_buf) {
    free(_jpg_buf);
}
_jpg_buf = NULL;

if (res != ESP_OK) break;
}
return res;
}

// ---- Single JPEG snapshot endpoint (/capture) ----
static esp_err_t capture_handler(httpd_req_t *req) {
    camera_fb_t *fb = esp_camera_fb_get();
    if (!fb) {
        httpd_resp_send_500(req);
        return ESP_FAIL;
    }

    esp_err_t res = ESP_OK;
    uint8_t *jpg_buf = NULL;
    size_t jpg_len = 0;
    bool needs_free = false;

    if (fb->format == PIXFORMAT_JPEG) {
        jpg_buf = fb->buf;
        jpg_len = fb->len;
    } else {
        bool converted = frame2jpg(fb, 90, &jpg_buf, &jpg_len);
        esp_camera_fb_return(fb);
        fb = NULL;
        if (!converted) {
            httpd_resp_send_500(req);
            return ESP_FAIL;
        }
        needs_free = true;
    }

    httpd_resp_set_type(req, "image/jpeg");
    httpd_resp_set_hdr(req, "Content-Disposition", "inline; filename=capture.jpg");
    res = httpd_resp_send(req, (const char *)jpg_buf, jpg_len);

    if (fb) esp_camera_fb_return(fb);
    if (needs_free && jpg_buf) free(jpg_buf);

    return res;
}

// ---- Manual test-alert endpoint (/trigger) ----
static esp_err_t trigger_handler(httpd_req_t *req) {
    triggerDeterrent();
    httpd_resp_set_type(req, "text/plain");
    httpd_resp_send(req, "Alert triggered", HTTPD_RESP_USE_STRLEN);
    return ESP_OK;
}

// ---- Root page: full-size stream + Capture/Test Alert buttons ----
static esp_err_t index_handler(httpd_req_t *req) {
    static const char PAGE[] =
        "<!DOCTYPE html><html><head><meta name='viewport' content='width=device-width,initial-scale=1'>"
        "<title>Animal Deterrent Cam</title>"
        "<style>"

```

```

"body{margin:0;background:#111;font-family:sans-serif;color:#fff;text-align:center}"
"img{display:block;width:100vw;height:80vh;object-fit:contain;background:#000;image-rendering:pixelated}"
"button{padding:14px 22px;margin:10px;font-size:16px;border:none;border-radius:8px;cursor:pointer}"
".cap{background:#2b8aef;color:#fff}"
".alert{background:#e74c3c;color:#fff}"
"#msg{min-height:20px}"
"</style></head><body>"
"<h2>ESP32-CAM Animal Deterrent</h2>"
"<img src='/stream'>"
"<div>"
"<button class='cap' onclick=\"window.open('/capture','_blank')\">Capture Photo</button>"
"<button class='alert' onclick=\"fetch('/trigger').then(()=>{document.getElementById('msg').innerText='Alert triggered!';})\">Alert</button>"
"</div>"
"<div id='msg'></div>"
"</body></html>";

httpd_resp_set_type(req, "text/html");
return httpd_resp_send(req, PAGE, HTTPD_RESP_USE_STRLEN);
}

void startCameraServer() {
    httpd_config_t config = HTTPD_DEFAULT_CONFIG();
    config.server_port = 81;
    config.max_uri_handlers = 8;

    httpd_uri_t stream_uri = {
        .uri = "/stream",
        .method = HTTP_GET,
        .handler = stream_handler,
        .user_ctx = NULL
    };
    httpd_uri_t capture_uri = {
        .uri = "/capture",
        .method = HTTP_GET,
        .handler = capture_handler,
        .user_ctx = NULL
    };
    httpd_uri_t trigger_uri = {
        .uri = "/trigger",
        .method = HTTP_GET,
        .handler = trigger_handler,
        .user_ctx = NULL
    };
    httpd_uri_t index_uri = {
        .uri = "/",
        .method = HTTP_GET,
        .handler = index_handler,
        .user_ctx = NULL
    };

    if (httpd_start(&stream_httpd, &config) == ESP_OK) {
        httpd_register_uri_handler(stream_httpd, &index_uri);
        httpd_register_uri_handler(stream_httpd, &stream_uri);
        httpd_register_uri_handler(stream_httpd, &capture_uri);
        httpd_register_uri_handler(stream_httpd, &trigger_uri);
    }
}

// ---- Sound buzzer + send Blynk alert ----
void triggerDeterrent() {
    digitalWrite(BUZZER_PIN, HIGH);
    if (Blynk.connected()) {
        Blynk.logEvent("animal_alert", "Alert! Target detected via ESP32-CAM.");
    }
    delay(3000);
    digitalWrite(BUZZER_PIN, LOW);
}

void setup() {
    Serial.begin(115200);
    Serial.setDebugOutput(true);
    Serial.println();

    pinMode(BUZZER_PIN, OUTPUT);
    digitalWrite(BUZZER_PIN, LOW);

    camera_config_t config;
    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer = LEDC_TIMER_0;
    config.pin_d0 = Y2_GPIO_NUM;
    config.pin_d1 = Y3_GPIO_NUM;
    config.pin_d2 = Y4_GPIO_NUM;
    config.pin_d3 = Y5_GPIO_NUM;
    config.pin_d4 = Y6_GPIO_NUM;
    config.pin_d5 = Y7_GPIO_NUM;
    config.pin_d6 = Y8_GPIO_NUM;
    config.pin_d7 = Y9_GPIO_NUM;
    config.pin_xclk = XCLK_GPIO_NUM;

```

```

config.pin_pclk      = PCLK_GPIO_NUM;
config.pin_vsync     = VSYNC_GPIO_NUM;
config.pin_href      = HREF_GPIO_NUM;
config.pin_sccb_sda  = SIOD_GPIO_NUM;
config.pin_sccb_scl  = SIOC_GPIO_NUM;
config.pin_pwdn      = PWDN_GPIO_NUM;
config.pin_reset     = RESET_GPIO_NUM;
config.xclk_freq_hz = 200000000;

// Proven-working settings for this sensor (JPEG mode failed with 0x106,
// RGB565 initializes reliably; we convert to JPEG in software for the stream)
config.pixel_format = PIXFORMAT_RGB565;
config.frame_size   = FRAMESIZE_QQVGA;
config.grab_mode     = CAMERA_GRAB_WHEN_EMPTY;
config.fb_location   = CAMERA_FB_IN_DRAM;
config.fb_count      = 1;

esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
    Serial.printf("Camera init failed with error 0x%x", err);
    return;
}

sensor_t *s = esp_camera_sensor_get();
if (s) {
    if (s->id.PID == OV3660_PID) {
        s->set_vflip(s, 1);
        s->set_brightness(s, 1);
        s->set_saturation(s, -2);
    }
    s->set_framesize(s, FRAMESIZE_QQVGA);
}

WiFi.begin(ssid, password);
WiFi.setSleep(false);

Serial.print("WiFi connecting");
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println("");
Serial.println("WiFi connected");

// Connect Blynk using the WiFi connection we already have
Blynk.config(BLYNK_AUTH_TOKEN);
Blynk.connect();

startCameraServer();

Serial.print("Camera Ready! Use 'http://");
Serial.print(WiFi.localIP());
Serial.println(":81/stream' to view");
}

void loop() {
    Blynk.run();

    // Type TRIGGER_ALERT in Serial Monitor to test the deterrent manually.
    // Later this can be replaced with a PIR sensor condition instead.
    if (Serial.available() > 0) {
        String command = Serial.readStringUntil('\n');
        command.trim();
        if (command == "TRIGGER_ALERT") {
            triggerDeterrent();
        }
    }
}

```

File: CameraWebServer_AnimalDeterrent.ino

6. Web Interface (Endpoints)

URL	Function
http://<ESP32-IP>:81/	Full-screen live stream page with Capture Photo and Test Alert buttons
http://<ESP32-IP>:81/stream	Raw MJPEG live video stream
http://<ESP32-IP>:81/capture	Single high-quality JPEG snapshot (also used by the Python script)
http://<ESP32-IP>:81/trigger	Manually sounds the buzzer and sends a Blynk alert

The ESP32-CAM's IP address is printed in the Arduino Serial Monitor (115200 baud) after it connects to WiFi.

7. AI-based Detection & Email Alerts (Python)

The ESP32-CAM alone does not have enough processing power for reliable person/animal recognition. Instead, a Python script running on a PC (on the same WiFi network) periodically fetches a photo from the ESP32-CAM's /capture endpoint and runs it through YOLOv8 — a free, pre-trained object-detection model that already recognises 80+ object classes (including person, dog, cat, cow, horse, bird, elephant, etc.) without any custom training.

7.1 Install Python and required libraries

Requires Python 3.9 or newer. Install the dependencies with:

```
pip install -r requirements.txt
```

requirements.txt:

```
ultralytics  
opencv-python  
requests  
numpy
```

7.2 Set up a Gmail App Password

- 1 Enable 2-Step Verification on the Gmail account (Google Account > Security).
- 2 Go to the "App Passwords" section and generate a new 16-character app password.
- 3 Use this app password in the script — NOT the normal Gmail login password.

7.3 Configure the script

Open detect_and_alert.py and fill in the CONFIG section: ESP32_CAM_URL (the ESP32-CAM's /capture URL), SENDER_EMAIL, SENDER_APP_PASSWORD, and RECEIVER_EMAIL.

7.4 Full Python Code (detect_and_alert.py)

```
"""
Animal / Unknown Person Detector + Email Alert
-----
Fetches photos from your ESP32-CAM, runs them through a pre-trained
YOLO object-detection model (no training needed), and emails you an
alert with the photo attached whenever a person or animal is seen.

SETUP (one time):
1. pip install -r requirements.txt
2. Fill in the CONFIG section below (ESP32 IP, Gmail address, Gmail App Password)
3. Run: python detect_and_alert.py
4. Leave it running in the background (same WiFi as the ESP32-CAM)
"""

import time
from datetime import datetime

import cv2
import numpy as np
import requests
from ultralytics import YOLO

import smtplib
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from email.mime.base import MIMEBase
from email import encoders

# =====
# CONFIG - edit these values
# =====
ESP32_CAM_URL = "http://10.135.138.126:81/capture" # <-- your ESP32-CAM IP

CHECK_INTERVAL_SECONDS = 5 # how often to check for a new photo
ALERT_COOLDOWN_SECONDS = 120 # don't send another email for 2 min after one is sent

SENDER_EMAIL = "your_email@gmail.com" # <-- Gmail address to SEND from
SENDER_APP_PASSWORD = "your_16_char_app_password" # <-- Gmail App Password (NOT your normal password)
RECEIVER_EMAIL = "your_email@gmail.com" # <-- where alerts should be sent (can be same as sender)

# Detected object labels that should trigger an alert (from the COCO dataset,
# which the pre-trained model already knows - no training required)
ANIMAL_CLASSES = {"cat", "dog", "horse", "sheep", "cow", "elephant",
                  "bear", "zebra", "giraffe", "bird"}
PERSON_CLASS = "person"
TARGET_CLASSES = ANIMAL_CLASSES | {PERSON_CLASS}

CONFIDENCE_THRESHOLD = 0.5 # 0.0-1.0, higher = fewer false alarms but may miss some
# =====

def load_model():
    print("Loading detection model (first run downloads it automatically)...")
    return YOLO("yolov8n.pt")

def fetch_frame():
    try:
        resp = requests.get(ESP32_CAM_URL, timeout=5)
        if resp.status_code == 200:
            img_array = np.frombuffer(resp.content, dtype=np.uint8)
            frame = cv2.imdecode(img_array, cv2.IMREAD_COLOR)
            return frame
        print(f"Camera returned status {resp.status_code}")
    except Exception as e:
        print("Could not reach ESP32-CAM:", e)
    return None

def send_email_alert(frame, detected_label):
    filename = f"alert_{datetime.now().strftime('%Y%m%d_%H%M%S')}.jpg"
    cv2.imwrite(filename, frame)

    msg = MIMEMultipart()
    msg["From"] = SENDER_EMAIL
    msg["To"] = RECEIVER_EMAIL
    msg["Subject"] = f"Alert: {detected_label} detected"

    body = (f"A '{detected_label}' was detected by your ESP32-CAM at "
           f"{datetime.now().strftime('%Y-%m-%d %H:%M:%S')}.")
    msg.attach(MIMEText(body, "plain"))

    with open(filename, "rb") as f:
        part = MIMEBase("application", "octet-stream")
        part.set_payload(f.read())
```

```

encoders.encode_base64(part)
part.add_header("Content-Disposition", f"attachment; filename={filename}")
msg.attach(part)

try:
    server = smtplib.SMTP("smtp.gmail.com", 587)
    server.starttls()
    server.login(SENDER_EMAIL, SENDER_APP_PASSWORD)
    server.send_message(msg)
    server.quit()
    print(f"Email sent: {detected_label}")
except Exception as e:
    print("Email send failed:", e)

def main():
    model = load_model()
    last_alert_time = 0
    print("Watching for animals / people... (Ctrl+C to stop)")

    while True:
        frame = fetch_frame()

        if frame is not None:
            results = model(frame, verbose=False)
            detected_labels = set()

            for r in results:
                for box in r.bboxes:
                    conf = float(box.conf[0])
                    if conf < CONFIDENCE_THRESHOLD:
                        continue
                    label = model.names[int(box.cls[0])]
                    if label in TARGET_CLASSES:
                        detected_labels.add(label)

            if detected_labels:
                now = time.time()
                if now - last_alert_time > ALERT_COOLDOWN_SECONDS:
                    label_str = ", ".join(sorted(detected_labels))
                    print(f"[{datetime.now().strftime('%H:%M:%S')}] Detected: {label_str}")
                    send_email_alert(frame, label_str)
                    last_alert_time = now
                else:
                    print(f"Detected {detected_labels} but still in cooldown, skipping email.")

            time.sleep(CHECK_INTERVAL_SECONDS)

if __name__ == "__main__":
    main()

```

File: detect_and_alert.py

Note: This detects a person or animal in general — it does not distinguish a known family member from an unfamiliar person. That would require adding face recognition as a separate step.

8. Running the Complete Project — Step by Step

Step	Task	Details
1	Wire the hardware	Connect the buzzer to GPIO 13 / GND as per Section 3.2. Connect the FTDI programmer for up
2	Upload the ESP32 code	Open CameraWebServer_AnimalDeterrent.ino in Arduino IDE, fill in Blynk credentials and WiFi
3	Switch to run mode	Remove the GPIO0-GND jumper, press RESET. Open Serial Monitor (115200 baud) and note t
4	Test the web page	On a device on the same WiFi, open http://<IP>:81/ — confirm the live stream, Capture, and Te
5	Test Blynk alerts	Press "Test Alert" on the web page (or send TRIGGER_ALERT in Serial Monitor) and confirm a
6	Set up the Python script	Install Python, run pip install -r requirements.txt, fill in the CONFIG section (ESP32 IP, Gmail ad
7	Run the detection script	Run: python detect_and_alert.py — leave it running in the background on a PC on the same Wi
8	Test end-to-end	Stand in front of the camera (or show an animal photo/video). Within a few seconds, an email w
9	Deploy in the field	Mount the ESP32-CAM in a weatherproof enclosure with a stable 5V power source (adapter or p

9. Troubleshooting Reference

Common issues encountered and resolved during development of this project:

Error	Cause	Fix
"This code is intended to run on the ESP8266 platform"	ESP32 platform selected in Arduino IDE	Tools > Board > esp32 > AI Thinker ESP32-CAM
"Please specify BLYNK_TEMPLATE_ID and define BLYNK_API for platform" / "Please specify BLYNK_TEMPLATE_ID and define BLYNK_API for platform"	Blynk #define lines at the very top of the file, before any other code	
"esp_camera.h: No such file or directory"	Board not set to an ESP32 variant	Select the correct ESP32 board (see above); header becomes esp32_camera.h
"HTTPD_GET was not declared in this scope"	ESP32 core (v3.x) renamed Use HTTP instead of HTTPD	Use <code>HTTP</code> instead of <code>HTTPD</code>
"Camera init failed with error 0x106"	SD card not initialised in <code>init()</code>	Some FR/2048 Size QVGA for combining
"board_config.h: No such file or directory"	Only the <code>main.cpp</code> is present in the file	Use the <code>main.cpp</code> file from the official (See the sample has more details)
Video appears tiny in the browser	Native camera resolution (QQVGA, 160x120) is displayed with the pixel size	It's styled to an explicit <code>width: 100%; height: 100%;</code>
Blynk "Video Streaming" widget shows a black screen	The black screen is not a video, it's just a black image	Make sure the video is not a black image, it's a video file, instead of a black image

End of Report